

Inventory Module – Final Comprehensive Specification.

1. Stock Level Controls

- Min Level / Max Level / Reorder Level configurable per item & store.
- Auto Reorder Suggestion when stock $\leq$ reorder.
- Excess Handling: If GRN or PO pushes stock above max $\rightarrow$ requires reason + higher approval.

2. Purchase & Approval Workflow

- Purchase Requisition $\rightarrow$ PO $\rightarrow$ GRN $\rightarrow$ Invoice.
- Approval Levels: L1 / L2 / L3 based on item category, value threshold, department policy.
- Price Variance: If purchase price deviates $>$ threshold (%), system:
 - Blocks auto-approval
 - Requires higher approval
 - Logs reason

3. Vendor & Purchasing Management

- **Purchasing Sources**
 - Open Market (urgent/local cash buys)
 - Cash Purchases (petty cash with receipt)
 - Credit Purchases (vendor invoice, AP aging)
 - Only Approved Vendors can be used in system
- **Vendor Master (Details Captured)**
 - Vendor ID / Code
 - Company Name, Contact Person, Phone, Email, Address
 - Categories / Items Approved
 - Payment Terms: Cash / Bank Transfer / Credit
 - Credit Days (e.g., Net 15, Net 30)
 - Excess Charges on Credit (late fee, service charge)
 - Contract Terms & Attachments (agreements, certificates)
 - Bank Details (IBAN, Account Title, Branch)
 - Vendor Rating (on-time delivery, quality acceptance, price stability)
 - Status: Pending / Approved / Blocked

- **Vendor Approval Workflow**
 - New vendor requests → reviewed by Finance + Management.
 - Only Approved vendors can be linked to PO.
 - Blocked vendors cannot be used until reinstated.
- **Purchasing Controls**
 - Vendor must be approved to raise PO.
 - System shows last purchase price & variance.
 - Payment terms auto-fill from Vendor Master.
 - Cash purchases logged with receipt photo.
 - Credit purchases linked to AP with due dates.
 - Excess charges logged separately for transparency.
- **Reports**
 - Vendor List (with status, categories, terms)
 - Purchases by Vendor (daily/weekly/monthly)
 - Credit Aging Report
 - Vendor Price Variance Report
 - Vendor Performance Report

4. Department Requisitions & Issues

- **Restaurant-specific:** Issues to Continental, Chinese, BBQ, Tandoor, Beverages, Dessert, etc.
- **General Organization:** Issues to Education, Administration, Construction, Kitchen, Security, Maintenance, Janitorial.
- Requisition → Issue Slip workflow with approval log.
- FEFO enforced if expiry tracking enabled.

5. Recipes & Costing Link

- If recipes exist, system auto-calculates:
 - Ideal consumption = Sales × Recipe ingredients
 - Actual consumption = Opening + Issues – Closing
 - Variance Report = Actual – Ideal (qty & %)
- Food Cost % auto-generated daily, weekly, monthly.
- Variance above tolerance → flagged.

6. Item Location & Storage

- For General Stores: Zone → Aisle → Rack → Bin mapping.
- Physical location searchable in system.
- Stat/Par Levels: Department-wise standard stock maintained.

7. Expiry & Warranty

- Expiry Tracking per batch (expiry date, shelf life).
- Alerts: Near expiry (30/60 days), Expired items → quarantine.
- Warranty Tracking for tools/equipment (expiry date + vendor).
- Warranty Alerts: Auto-notify before end of coverage.

8. Reusable / Saleable Items

- Used Oil, Scrap Tins, Packaging Material marked as reusable/saleable.
- Store → Market Sale transaction reduces stock and posts AR entry.
- Reports on margins and buyer details.

9. Theft & Physical Checks

- Theft Report: Adjustment entry with category “Theft”, incident reference, optional photos, approval required.
- Physical Stock Check: Scheduled cycle/full counts.
- Variance auto-posted with approval.
- Physical Check Report: system vs actual, value variance, reasons.

10. Store Staff KPI & Donation Tracking

- **Entries Count:** Track number of system entries per user (GRN, Issues, Transfers, Adjustments, Sales, etc.).
- **Working Hours & Overtime:** Expected hours vs actual login/logout; overtime flagged and logged.
- **Utilization Dashboard:** Transactions handled vs expected workload, top performers vs idle staff.
- **Donation GRN:** Capture Donor Name, Donation Type, Items, Qty, Market Value, Date.
- **Automatic Donor Record Update:** Each donation updates donor profile in system.
- **Donation Frequency Report:** Donor-wise, item-wise, value-wise, frequency.
- **Traceability:** Donations flow into inventory, consumption, and reporting.

11. Reporting

Frequency: Daily, Weekly, Monthly, Quarterly, Custom.

Reports include:

- Overall Organization Store Report
- Department Requisitions & Issues
- Vendor List & Purchases by Vendor
- Credit Aging Report
- Price Variance Report
- Expiry/Warranty Alerts
- Recipe Variance & Food Cost %
- Excess Stock Report (above max)
- Theft & Adjustment Report
- Physical Stock Check Report
- Reusable/Scrap Sales Report
- Par/Stat Level Compliance
- Batch Ageing Report
- User KPI Report
- Donation Register
- Vendor Performance Report

12. Alerts & Automation

- Min/Reorder/Max Stock Alerts
- Price Variance Alerts
- Expiry/Warranty Alerts
- Recipe Variance Alerts
- Slow/Inactive Items Alerts
- Loss/Theft Alerts
- Donation Alerts (new donation, donor frequency)
- Credit Due/Overdue Alerts

13. Roles & Permissions

- Store Keeper: GRN, Issues, Adjustments, Stock Count
- Department Head: Requisitions, Issue Acknowledgement
- Purchasing Officer: PO creation
- Approver L1–L3: Based on thresholds
- Finance: Invoices, Variance Review
- Admin: Master data, thresholds, approvals
- Auditor: Read-only reports, audit logs

14. Test Cases for QA

- Item below reorder → system generates suggestion
- GRN pushes above max → requires reason + L2/L3 approval
- PO price variance > threshold → higher approval required
- Expiry items issued → system enforces FEFO
- Warranty alert before end date
- Department issues logged correctly (restaurant vs general)
- Recipe variance flagged for out-of-tolerance items
- Store sale (used oil) generates AR entry + reduces stock
- Theft report creates adjustment + incident log
- Physical stock check variance posted to ledger
- User KPI report shows entries count & utilization
- Donation register shows donor, items, value, frequency
- Vendor approval required before PO creation
- Credit Aging Report matches Finance AP records

SQL Server DDL (Data Definition Language) Script

Here is the full CREATE TABLE script, organized by functional module.

```
=====
-- SECTION 1: CORE MASTERS (Company, Stores, Users, Permissions, Departments)
-- These tables form the foundational structure for multi-tenancy and user access.
=====
```

```
CREATE TABLE Companies (
```

```
    CompanyID INT PRIMARY KEY IDENTITY(1,1),
```

```
    CompanyName NVARCHAR(255) NOT NULL,
```

```
    IsActive BIT NOT NULL DEFAULT 1
```

```
);
```

```
CREATE TABLE Stores (
```

```
    StoreID INT PRIMARY KEY IDENTITY(1,1),
```

```
    CompanyID INT NOT NULL FOREIGN KEY REFERENCES Companies(CompanyID),
```

```
    StoreName NVARCHAR(100) NOT NULL,
```

```
    LocationAddress NVARCHAR(MAX),
```

```
    IsActive BIT NOT NULL DEFAULT 1
```

```
);
```

```
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY IDENTITY(1,1),
    StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),
    DepartmentName NVARCHAR(100) NOT NULL UNIQUE
);
```

```
CREATE TABLE Roles (
    RoleID INT PRIMARY KEY IDENTITY(1,1),
    RoleName NVARCHAR(50) NOT NULL UNIQUE
);
```

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY IDENTITY(1,1),
    Username NVARCHAR(100) NOT NULL UNIQUE,
    HashedPassword NVARCHAR(255) NOT NULL,
    Email NVARCHAR(100) NOT NULL UNIQUE,
    FirstName NVARCHAR(100),
    LastName NVARCHAR(100),
    RoleID INT NOT NULL FOREIGN KEY REFERENCES Roles(RoleID),
    StoreID INT FOREIGN KEY REFERENCES Stores(StoreID), -- User's primary store
    IsActive BIT NOT NULL DEFAULT 1
);
```

```
=====
-- SECTION 2: ITEM & VENDOR MASTERS
-- Manages all items, their categorization, and vendor information.
=====
```

```
CREATE TABLE ItemCategories (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(100) NOT NULL,
    ParentCategoryID INT NULL FOREIGN KEY REFERENCES ItemCategories(CategoryID)
);
```

```
CREATE TABLE Items (
    ItemID INT PRIMARY KEY IDENTITY(1,1),
    SKU NVARCHAR(50) NOT NULL UNIQUE,
```

```

    ItemName NVARCHAR(255) NOT NULL,
    Description NVARCHAR(MAX),
    CategoryID INT NOT NULL FOREIGN KEY REFERENCES ItemCategories(CategoryID),
    UnitOfMeasure NVARCHAR(50) NOT NULL,
    IsTrackedByBatch BIT NOT NULL DEFAULT 0, -- Critical for FEFO
    IsWarrantyTracked BIT NOT NULL DEFAULT 0,
    IsSaleable BIT NOT NULL DEFAULT 1, -- For regular items and scrap
    IsReusable BIT NOT NULL DEFAULT 0 -- For tracking things like packaging
);

```

```

CREATE TABLE Vendors (
    VendorID INT PRIMARY KEY IDENTITY(1,1),
    VendorCode NVARCHAR(50) NOT NULL UNIQUE,
    CompanyName NVARCHAR(255) NOT NULL,
    ContactPerson NVARCHAR(100),
    Phone NVARCHAR(50),
    Email NVARCHAR(100),
    Address NVARCHAR(MAX),
    PaymentTerms NVARCHAR(50) NOT NULL, -- 'Cash', 'Credit', 'Bank Transfer'
    CreditDays INT NULL,
    Status NVARCHAR(50) NOT NULL CHECK (Status IN ('Pending', 'Approved', 'Blocked')),
    -- Audit fields
    CreatedBy INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE()
);

```

```

CREATE TABLE VendorItemMap (
    VendorItemMapID INT PRIMARY KEY IDENTITY(1,1),
    VendorID INT NOT NULL FOREIGN KEY REFERENCES Vendors(VendorID),
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),
    LastPurchasePrice DECIMAL(18, 4) NULL,
    CONSTRAINT UQ_Vendor_Item UNIQUE (VendorID, ItemID)
);

```

```
-- =====  
-- SECTION 3: INVENTORY STOCK & LOCATION ENGINE  
-- The heart of the inventory system, tracking stock levels, batches, and physical locations.  
=====
```

```
CREATE TABLE ItemStockLevels (  
    ItemStockLevelID INT PRIMARY KEY IDENTITY(1,1),  
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),  
    StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),  
    QuantityOnHand DECIMAL(18, 4) NOT NULL DEFAULT 0,  
    MinLevel DECIMAL(18, 4) NOT NULL DEFAULT 0,  
    MaxLevel DECIMAL(18, 4) NOT NULL DEFAULT 0,  
    ReorderLevel DECIMAL(18, 4) NOT NULL DEFAULT 0,  
    CONSTRAINT UQ_Item_Store_Stock UNIQUE (ItemID, StoreID)  
);
```

```
CREATE TABLE ItemBatches (  
    BatchID INT PRIMARY KEY IDENTITY(1,1),  
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),  
    StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),  
    BatchNumber NVARCHAR(100) NOT NULL,  
    Quantity DECIMAL(18, 4) NOT NULL,  
    ReceivedDate DATETIME2 NOT NULL DEFAULT GETUTCDATE(),  
    ExpiryDate DATE NULL,  
    WarrantyEndDate DATE NULL,  
    CONSTRAINT UQ_Item_Store_Batch UNIQUE (ItemID, StoreID, BatchNumber)  
);
```

```
-- The immutable ledger for full auditability
```

```
CREATE TABLE InventoryLedger (  
    LedgerID BIGINT PRIMARY KEY IDENTITY(1,1),  
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),  
    StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),  
    BatchID INT NULL FOREIGN KEY REFERENCES ItemBatches(BatchID),  
    TransactionType NVARCHAR(50) NOT NULL, -- e.g., 'GRN', 'Issue', 'Sale', 'Adjustment'  
    TransactionID BIGINT NOT NULL, -- FK to the source document (e.g., GRN_ID)  
    ChangeInQuantity DECIMAL(18, 4) NOT NULL, -- Positive for additions, negative for  
deductions
```



```
NewQuantityOnHand DECIMAL(18, 4) NOT NULL, -- The resulting quantity after the
transaction
```

```
TransactionDate DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
UserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID)
);
```

```
CREATE TABLE ItemLocations (
    LocationID INT PRIMARY KEY IDENTITY(1,1),
    StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),
    Zone NVARCHAR(50),
    Aisle NVARCHAR(50),
    Rack NVARCHAR(50),
    Bin NVARCHAR(50),
    CONSTRAINT UQ_Location UNIQUE (StoreID, Zone, Aisle, Rack, Bin)
);
```

```
CREATE TABLE BatchLocationMap (
    BatchLocationMapID BIGINT PRIMARY KEY IDENTITY(1,1),
    BatchID INT NOT NULL FOREIGN KEY REFERENCES ItemBatches(BatchID),
    LocationID INT NOT NULL FOREIGN KEY REFERENCES ItemLocations(LocationID),
    Quantity DECIMAL(18, 4) NOT NULL
);
```

```
=====
-- SECTION 4: PROCUREMENT WORKFLOW (PROCURE-TO-PAY)
-- Manages the entire lifecycle from requisition to invoice.
=====
```

```
CREATE TABLE PurchaseRequisitions (
    RequisitionID INT PRIMARY KEY IDENTITY(1,1),
    DepartmentID INT NOT NULL FOREIGN KEY REFERENCES Departments(DepartmentID),
    Status NVARCHAR(50) NOT NULL DEFAULT 'Pending',
    RequestedByUserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE()
);
```

```
CREATE TABLE PurchaseRequisitionItems (
    RequisitionItemID INT PRIMARY KEY IDENTITY(1,1),
```

```

    RequisitionID INT NOT NULL FOREIGN KEY REFERENCES
PurchaseRequisitions(RequisitionID),
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),
    Quantity DECIMAL(18, 4) NOT NULL
);

```

```

CREATE TABLE PurchaseOrders (
    PO_ID INT PRIMARY KEY IDENTITY(1,1),
    VendorID INT NOT NULL FOREIGN KEY REFERENCES Vendors(VendorID),
    StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),
    Status NVARCHAR(50) NOT NULL DEFAULT 'Pending',
    CreatedByUserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    TotalAmount DECIMAL(18, 2)
);

```

```

CREATE TABLE PurchaseOrderItems (
    PO_ItemID INT PRIMARY KEY IDENTITY(1,1),
    PO_ID INT NOT NULL FOREIGN KEY REFERENCES PurchaseOrders(PO_ID),
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),
    Quantity DECIMAL(18, 4) NOT NULL,
    UnitPrice DECIMAL(18, 4) NOT NULL
);

```

```

CREATE TABLE GoodsReceivedNotes (
    GRN_ID BIGINT PRIMARY KEY IDENTITY(1,1),
    PO_ID INT NULL FOREIGN KEY REFERENCES PurchaseOrders(PO_ID), -- Can be null for
donations/direct GRN
    StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),
    ReceivedByUserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    ReceivedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    IsDonation BIT NOT NULL DEFAULT 0
);

```

```

CREATE TABLE GoodsReceivedNoteItems (
    GRN_ItemID BIGINT PRIMARY KEY IDENTITY(1,1),
    GRN_ID BIGINT NOT NULL FOREIGN KEY REFERENCES GoodsReceivedNotes(GRN_ID),
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),

```

```
BatchID INT NULL FOREIGN KEY REFERENCES ItemBatches(BatchID), -- Populated on  
receive if batch-tracked
```

```
QuantityReceived DECIMAL(18, 4) NOT NULL  
);
```

```
CREATE TABLE Invoices (  
    InvoiceID INT PRIMARY KEY IDENTITY(1,1),  
    VendorID INT NOT NULL FOREIGN KEY REFERENCES Vendors(VendorID),  
    GRN_ID BIGINT NOT NULL FOREIGN KEY REFERENCES GoodsReceivedNotes(GRN_ID),  
    InvoiceNumber NVARCHAR(100) NOT NULL,  
    InvoiceDate DATE NOT NULL,  
    DueDate DATE NOT NULL,  
    TotalAmount DECIMAL(18, 2) NOT NULL,  
    Status NVARCHAR(50) NOT NULL DEFAULT 'Unpaid'  
);
```

```
-- =====  
-- SECTION 5: INTERNAL STOCK MOVEMENT & CONSUMPTION  
-- Manages internal requisitions and issues between departments.  
=====
```

```
CREATE TABLE InternalRequisitions (  
    InternalRequisitionID INT PRIMARY KEY IDENTITY(1,1),  
    DepartmentID INT NOT NULL FOREIGN KEY REFERENCES Departments(DepartmentID),  
    Status NVARCHAR(50) NOT NULL DEFAULT 'Pending',  
    RequestedByUserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),  
    CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE()  
);
```

```
CREATE TABLE InternalRequisitionItems (  
    InternalRequisitionItemID INT PRIMARY KEY IDENTITY(1,1),  
    InternalRequisitionID INT NOT NULL FOREIGN KEY REFERENCES  
InternalRequisitions(InternalRequisitionID),  
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),  
    Quantity DECIMAL(18, 4) NOT NULL  
);
```

```
CREATE TABLE IssueSlips (  
    IssueSlipID INT PRIMARY KEY IDENTITY(1,1),  
    InternalRequisitionID INT NOT NULL FOREIGN KEY REFERENCES  
InternalRequisitions(InternalRequisitionID),  
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),  
    Quantity DECIMAL(18, 4) NOT NULL  
);
```

```

        IssueSlipID BIGINT PRIMARY KEY IDENTITY(1,1),
        InternalRequisitionID INT NOT NULL FOREIGN KEY REFERENCES
InternalRequisitions(InternalRequisitionID),
        StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),
        IssuedByUserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
        AcknowledgedByUserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
        IssuedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE()
    );

CREATE TABLE IssueSlipItems (
    IssueSlipItemID BIGINT PRIMARY KEY IDENTITY(1,1),
    IssueSlipID BIGINT NOT NULL FOREIGN KEY REFERENCES IssueSlips(IssueSlipID),
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),
    BatchID INT NOT NULL FOREIGN KEY REFERENCES ItemBatches(BatchID), -- Crucial for
FEFO
    QuantityIssued DECIMAL(18, 4) NOT NULL
);

-- =====
-- SECTION 6: RECIPES & COSTING
-- Manages recipe definitions for calculating ideal consumption and food cost.
=====

CREATE TABLE Recipes (
    RecipeID INT PRIMARY KEY IDENTITY(1,1),
    RecipeName NVARCHAR(255) NOT NULL,
    MenuItemID INT NULL, -- FK to your future Sales/POS MenuItem table
    Yield DECIMAL(18, 4) NOT NULL,
    YieldUnit NVARCHAR(50) NOT NULL
);

CREATE TABLE RecipeItems (
    RecipeItemID INT PRIMARY KEY IDENTITY(1,1),
    RecipeID INT NOT NULL FOREIGN KEY REFERENCES Recipes(RecipeID),
    IngredientItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),
    Quantity DECIMAL(18, 4) NOT NULL,
    UnitOfMeasure NVARCHAR(50) NOT NULL
);

```

```
=====
-- SECTION 7: AUDITING & QUALITY CONTROL
```

```
-- Tables for physical checks, adjustments, and the generic approval workflow.
=====
```

```
CREATE TABLE StockAdjustments (
    AdjustmentID BIGINT PRIMARY KEY IDENTITY(1,1),
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),
    StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),
    BatchID INT NULL FOREIGN KEY REFERENCES ItemBatches(BatchID),
    AdjustmentType NVARCHAR(50) NOT NULL, -- 'Theft', 'Spoilage', 'Correction'
    QuantityChange DECIMAL(18, 4) NOT NULL, -- Negative for loss, positive for gain
    Reason NVARCHAR(MAX) NOT NULL,
    IncidentReference NVARCHAR(100) NULL,
    Status NVARCHAR(50) NOT NULL DEFAULT 'Pending',
    CreatedByUserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE()
);
```

```
CREATE TABLE PhysicalStockChecks (
    StockCheckID INT PRIMARY KEY IDENTITY(1,1),
    StoreID INT NOT NULL FOREIGN KEY REFERENCES Stores(StoreID),
    CheckType NVARCHAR(50) NOT NULL, -- 'Cycle Count', 'Full Count'
    Status NVARCHAR(50) NOT NULL DEFAULT 'InProgress',
    CheckedByUserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    CheckDate DATE NOT NULL
);
```

```
CREATE TABLE PhysicalStockCheckItems (
    StockCheckItemID INT PRIMARY KEY IDENTITY(1,1),
```

```

    StockCheckID INT NOT NULL FOREIGN KEY REFERENCES
PhysicalStockChecks(StockCheckID),
    ItemID INT NOT NULL FOREIGN KEY REFERENCES Items(ItemID),
    SystemQuantity DECIMAL(18, 4) NOT NULL,
    PhysicalQuantity DECIMAL(18, 4) NOT NULL,
    Variance AS (PhysicalQuantity - SystemQuantity)
);

```

```

CREATE TABLE ApprovalWorkflows (
    ApprovalID INT PRIMARY KEY IDENTITY(1,1),
    DocumentType NVARCHAR(50) NOT NULL, -- 'PO', 'Requisition', 'Adjustment', 'Vendor'
    DocumentID BIGINT NOT NULL,
    ApprovalLevel INT NOT NULL, -- L1, L2, L3
    ApproverUserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    Status NVARCHAR(50) NOT NULL DEFAULT 'Pending',
    Comments NVARCHAR(MAX),
    Timestamp DATETIME2 NULL
);

```

```

=====
-- SECTION 8: SUPPORTING & VALUE-ADD MODULES
-- Tables for Donations, KPI Tracking, etc.
=====

```

```

CREATE TABLE Donors (
    DonorID INT PRIMARY KEY IDENTITY(1,1),
    DonorName NVARCHAR(255) NOT NULL,
    ContactInfo NVARCHAR(MAX)
);

```

```

CREATE TABLE Donations (
    DonationID BIGINT PRIMARY KEY IDENTITY(1,1),
    GRN_ID BIGINT NOT NULL FOREIGN KEY REFERENCES GoodsReceivedNotes(GRN_ID),
    DonorID INT NOT NULL FOREIGN KEY REFERENCES Donors(DonorID),
    DonationType NVARCHAR(100),
    MarketValue DECIMAL(18, 2)
);

```

```

CREATE TABLE UserActivityLogs (
    LogID BIGINT PRIMARY KEY IDENTITY(1,1),
    UserID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    ActivityType NVARCHAR(100) NOT NULL, -- 'GRN Created', 'Issue Slip Posted'
    EntityID BIGINT, -- e.g., The ID of the GRN or Issue Slip
    Timestamp DATETIME2 NOT NULL DEFAULT GETUTCDATE()
);

```

```

=====
-- SECTION 9: INDEXING FOR PERFORMANCE
=====

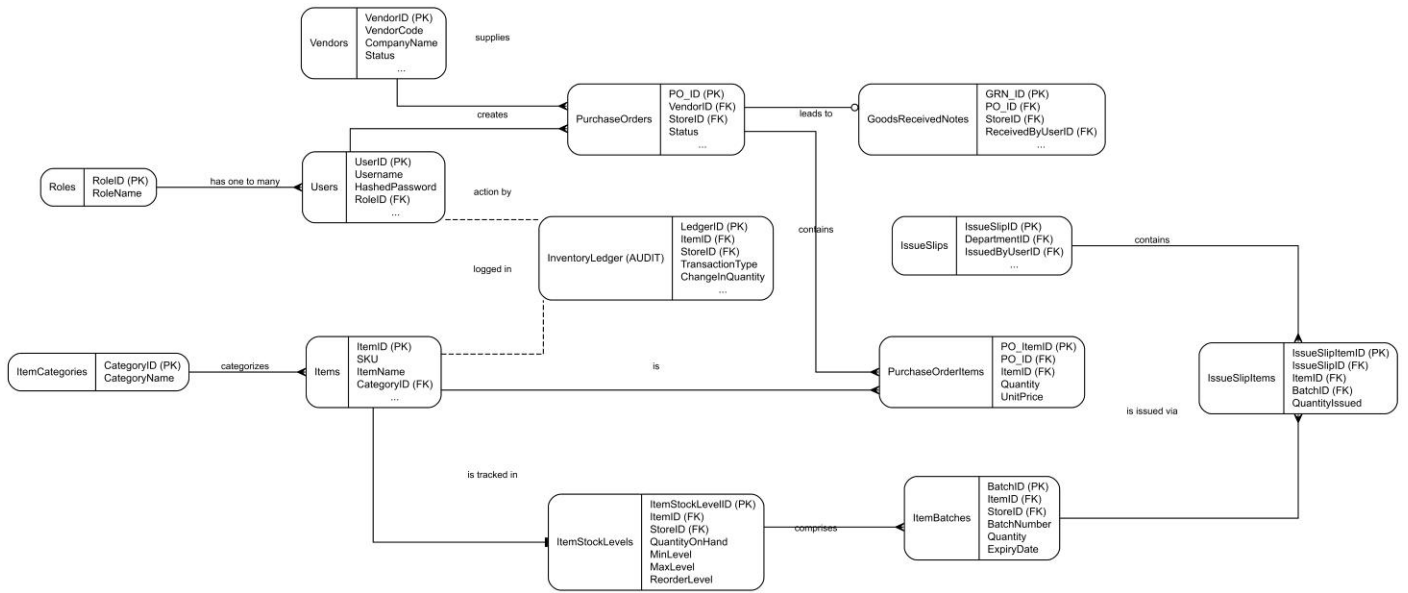
```

```

CREATE INDEX IX_Items_ItemName ON Items(ItemName);
CREATE INDEX IX_Vendors_CompanyName ON Vendors(CompanyName);
CREATE INDEX IX_InventoryLedger_ItemID_StoreID ON InventoryLedger(ItemID, StoreID);
CREATE INDEX IX_PurchaseOrders_VendorID_Status ON PurchaseOrders(VendorID, Status);
CREATE INDEX IX_IssueSlips_StoreID_IssuedAt ON IssueSlips(StoreID, IssuedAt);

```

1. Entity-Relationship Diagram (ERD) in DOT



2. Procure-to-Pay (P2P) Cycle Flowchart in DOT

